# Solution: Squaring a Sorted Array

## Problem Statement

Given a sorted array, create a new array containing squares of all the numbers of the input array in the sorted order.

**Example 1:**

```
Input: [-2, -1, 0, 2, 3]
Output: [0, 1, 4, 4, 9]
```

**Example 2:**

```
Input: [-3, -1, 0, 1, 2]
Output: [0, 1, 1, 4, 9]
```

## Solution

We can use a brute-force approach to iterate the input array and calculate the square of each number. We can store these squares in a new array and then sort the resulting array using any sorting algorithm like Quicksort or Mergesort. Because of the sorting, this approach has a time complexity of $O(N * logN)$, where $N$ is the length of the input array. Here is a Python solution for this approach:

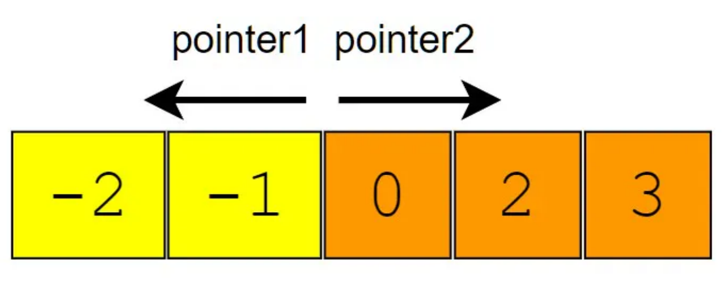
```
def sorted_squares(nums):
  return sorted([num**2 for num in nums])
```

Can we do better than this? Can we avoid sorting? Is it possible to generate the output in sorted order?

The tricky part is that we can have negative numbers in the input array, which makes it harder to generate the output array with squares in sorted order.

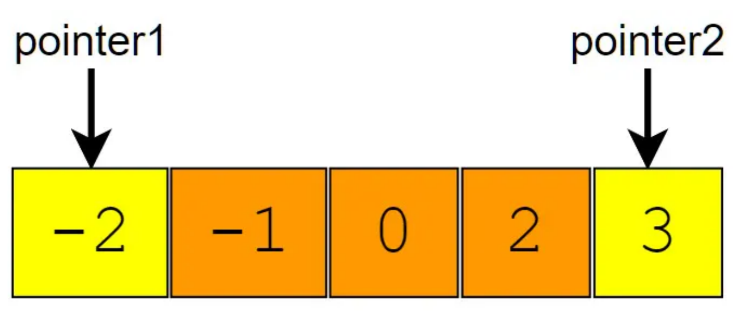
One easier approach could be to first locate the index of the first positive number in the input array. After that, we can utilize the **Two Pointers** technique to iterate over the array, with one pointer moving forward to scan positive numbers, and the other pointer moving backward to scan negative numbers. At each step, we can compare the squares of the numbers pointed by both pointers and append the smaller square to the output array.

For the above-mentioned Example-1, we will do something like this:



Since the numbers at both ends can give us the largest square, an alternate approach could be to use two pointers starting at both ends of the input array. At any step, whichever pointer gives us the bigger square, we add it to the result array and move to the next/previous number. Please note that we will be appending the bigger square (as opposed to the previous approach) because the two pointers are moving from larger squares to smaller squares. For that, we will be inserting the squares at the end of the output array.

For the above-mentioned Example-1, we will do something like this:

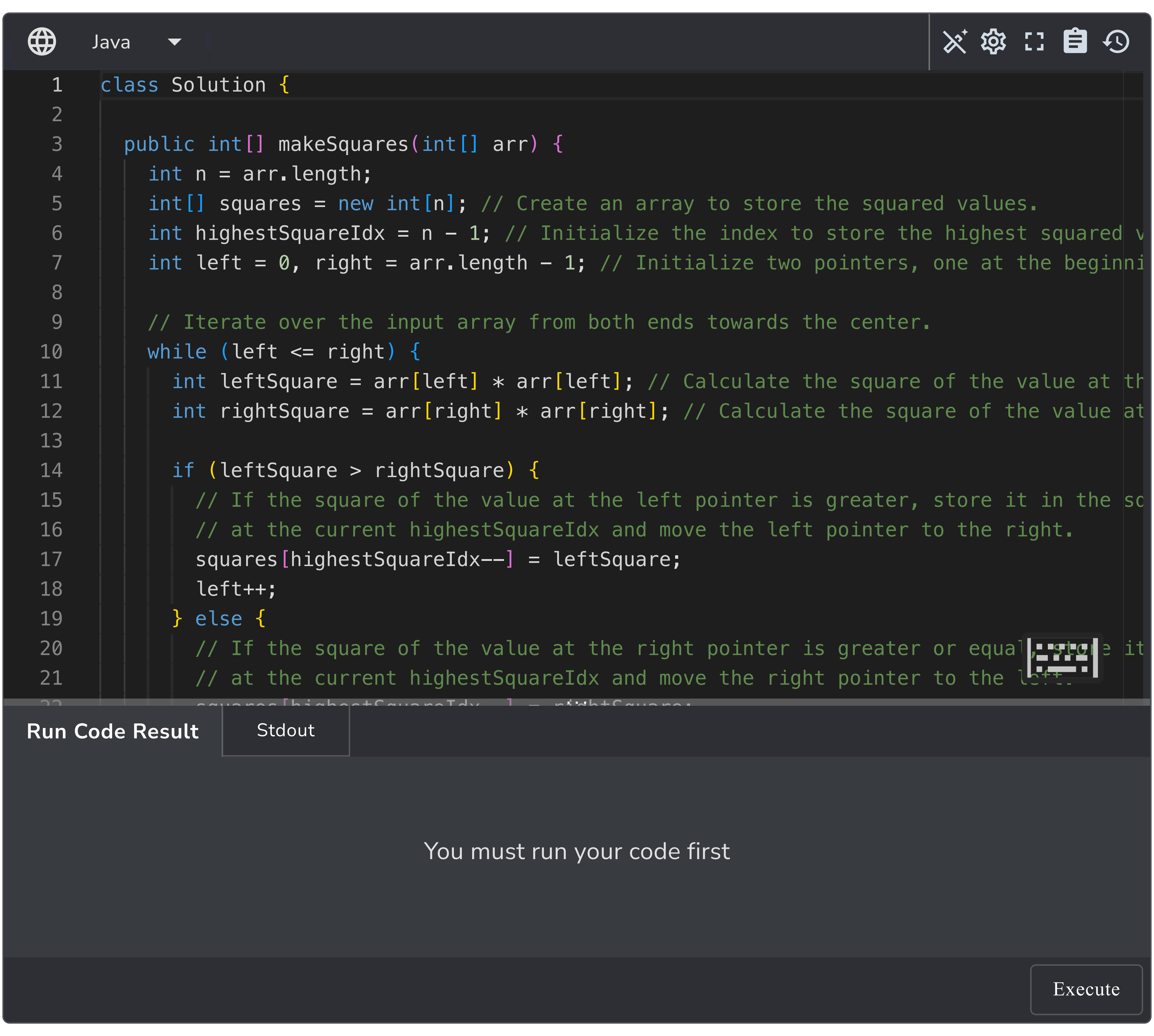


Here's a detailed walkthrough of the algorithm:

1. We start by obtaining the length of the input array, `arr`, which we store in variable `n`. Then, we create a new array, `squares`, of the same length to hold the squared values. We also create a variable `highestSquareIdx` and set it to `n - 1`, the last index of `squares`, which will help us populate the `squares` array from the highest (rightmost) index towards the lowest (leftmost).
2. We initialize two pointers, `left` and `right`, to 0 and `n - 1`, respectively. These pointers represent the indices of the elements at the start (lowest) and end (highest) of the array.
3. We enter a loop that continues as long as `left` is less than or equal to `right`.
4. In each iteration, we calculate the squares of the elements at the `left` and `right` indices, storing them in `leftSquare` and `rightSquare` respectively.
5. We then compare `leftSquare` with `rightSquare`. The larger of these two squares is inserted at the position of `highestSquareIdx` in the `squares` array, and `highestSquareIdx` is decremented.
6. If `leftSquare` was larger, we increment `left` to move towards the higher numbers in the array. If `rightSquare` was larger or equal, we decrement `right` to move towards the lower numbers in the array. We're comparing absolute square values, so even if numbers in the array are negative, we're dealing with their positive square.
7. This process repeats, filling up the `squares` array from right to left, until `left` and `right` meet or cross each other.
8. At this point, the `squares` array is filled with the squares of the numbers in the input array, sorted in ascending order. This array is then returned as the result.

## Code

Here is the code for the second approach discussed above:



```
class Solution {

  public int[] makeSquares(int[] arr) {
    int n = arr.length;
    int[] squares = new int[n]; // Create an array to store the squared values.
    int highestSquareIdx = n - 1; // Initialize the index to store the highest squared v
    int left = 0, right = arr.length - 1; // Initialize two pointers, one at the beginni

    // Iterate over the input array from both ends towards the center.
    while (left <= right) {
      int leftSquare = arr[left] * arr[left]; // Calculate the square of the value at th
      int rightSquare = arr[right] * arr[right]; // Calculate the square of the value at

      if (leftSquare > rightSquare) {
        // If the square of the value at the left pointer is greater, store it in the sc
        // at the current highestSquareIdx and move the left pointer to the right.
        squares[highestSquareIdx--] = leftSquare;
        left++;
      } else {
        // If the square of the value at the right pointer is greater or equal, store it
        // at the current highestSquareIdx and move the right pointer to the left.
```

## Time Complexity

The above algorithm's time complexity will be $O(N)$ as we are iterating the input array only once.

## Space Complexity

The above algorithm's space complexity will also be $O(N)$; this space will be used for the output array.

← Previous
Next →
Squaring A Sorted Array (Easy)
Triplet Sum To Zero (Medium)
☐ Mark as Completed